LPP 3CFU – Domande d'esame A.A. 25/26
Parte 4

Luca Roversi

December 12, 2025

CT0450 Natural transformations in Haskell

Exercise 1. Recall the definition of natural transformation between two functors $\mathcal{F}$ and $\mathcal{G}$. □

Exercise 2. Let the following two Haskell functors `Maybe` and `[]` be given. Consider the following Haskell function

```
maybeToList :: Maybe a -> [a]
maybeToList Nothing  = []
maybeToList (Just x) = [x]
```

Prove the naturality law in the case where the argument is `Just x`. □

Exercise 3. Let the following two Haskell functors `[]` and `Maybe` be given. Consider the following Haskell function

```
listToMaybe :: [a] -> Maybe a
listToMaybe []    = Nothing
listToMaybe (x:_) = Just x
```

Prove the naturality law in the case where the argument is `h:t`. □

Exercise 4. Let the following two Haskell functors `Maybe` and `Either ()` be given. Consider the following Haskell function

```
toEitherUnit :: Maybe a -> Either () a
toEitherUnit Nothing  = Left ()
toEitherUnit (Just x) = Right x
```

Prove the naturality law in the case where the argument is `Just x`. □

Exercise 5. Let the following two Haskell functors `Maybe` and `Either ()` be given. Consider the following Haskell function

```
toEitherUnit :: Maybe a -> Either () a
toEitherUnit Nothing  = Left ()
toEitherUnit (Just x) = Right x
```

Prove the naturality law in the case where the argument is `Nothing`. □

Exercise 6. Let the following two Haskell functors `Either ()` and `Maybe` be given. Consider the following Haskell function

```
fromEitherUnit :: Either () a -> Maybe a
fromEitherUnit (Left ()) = Nothing
fromEitherUnit (Right x) = Just x
```

Prove the naturality law in the case where the argument is `Right x`. □

Exercise 7. Let the following two Haskell functors `[]` and `Either ()` be given. Consider the following Haskell function

```
headEither :: [a] -> Either () a
headEither []    = Left ()
headEither (x:_) = Right x
```

Prove the naturality law in the case where the argument is `h:t`. □

Exercise 8. Let the following two Haskell functors `Either ()` and `[]` be given. Consider the following Haskell function

```
eitherToList :: Either () a -> [a]
eitherToList (Left ()) = []
eitherToList (Right x) = [x]
```

Prove the naturality law in the case where the argument is `Right x`. □

CT0530 Monad in Haskell

Exercise 9. Consider the applicative functor `Maybe a`.

1. Define the function `(>>=)` for the functor `Maybe a`.
2. Prove the associativity law for `(>>=)`. □

Exercise 10. Consider the applicative functor `Either u`.

1. Define the function `(>>=)` for the functor `Either u`.
2. Prove the associativity law for `(>>=)`. □

Exercise 11. Let us assume that we have defined the type:

```
data List a where
    Nil :: List a
    Cons :: a -> List a -> List a
```

and that we already proved that it is an applicative functor.

1. Define the function `(>>=)` for the functor `List a`.

2. Prove the associativity law for `(>>=)`. □

Exercise 12. Let us assume that we have defined the type:

```
newtype Diagonal a = PutD {getD :: (,) a a}
```

and that we already proved that it is an applicative functor.

1. Define the function `(>>=)` for `Diagonal a`.
2. Prove the associativity law for `(>>=)`. □

Exercise 13. Let us assume that we have defined the type:

```
newtype Const c a = Const { getConst :: c }
```

and that we already proved that it is an applicative functor.

1. Define the function `(>>=)` for the functor `Const c`.
2. Prove the associativity law for `(>>=)`. □

Exercise 14. Let us assume that we have defined the type:

```
newtype Identity a = Identity { runIdentity :: a }
```

and that we already proved that it is an applicative functor.

1. Define the function `(>>=)` for the functor `Identity a`.
2. Prove the associativity law for `(>>=)`. □

Exercise 15. Let us assume that we have defined the type:

```
newtype Reader r a where
    Reader :: (r -> a) -> Reader r a
```

and that we already proved that it is an applicative functor.

1. Define the function `(>>=)` for the functor `Reader r`.
2. Write the essential description that justifies the given definition of `(>>=)`. □

Exercise 16. Let us assume that we have defined the type:

```
newtype State s a = State { runState :: s -> (a, s) }
```

and that we already proved that it is an applicative functor.

1. Define the function `(>>=)` for the functor `State s`.
2. Write the essential description that justifies the given definition of `(>>=)`. □

Exercise 17. Let us assume that we have defined the type:

```
newtype State s a = State { runState :: s -> (a, s) }
```

1. Prove that it is a functor by defining the function `fmap`.
2. Write the essential description that justifies the given definition of `fmap`. □

Exercise 18. Let us assume that we have defined the type:

```
newtype State s a = State { runState :: s -> (a, s) }
```

and that we already proved that it is a functor.

1. Prove that it is applicative by defining the functions `pure` and `(<*>)`.
2. Write the essential description that justifies the given definition of `(<*>)`. □